Long-Run Workflows: Harness, Loop, and Graph

Hulai Zhang

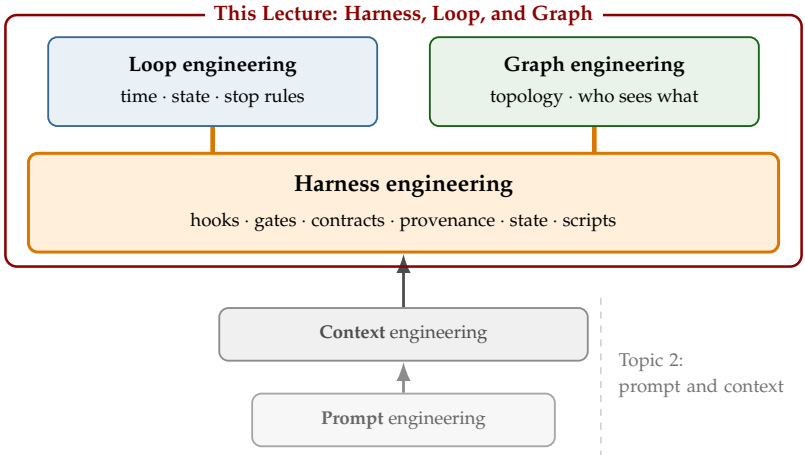
International School of Finance, Fudan University

Fall 2026

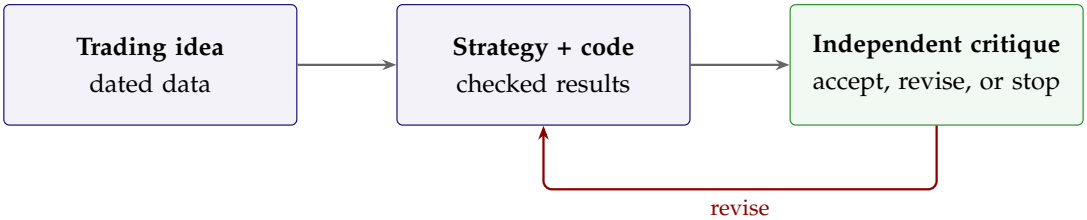


Engineering Ladder

Loop and Graph stand on the harness; they are not further rungs above it.

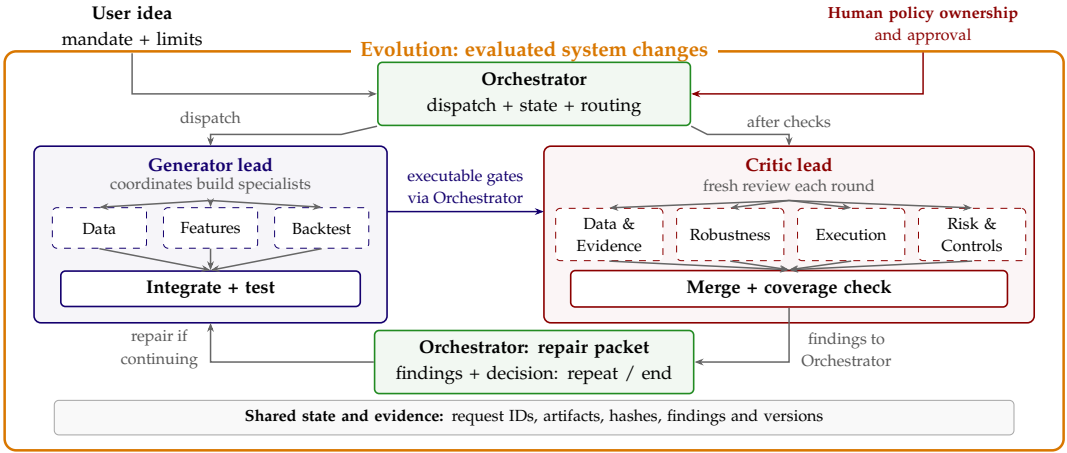


Build the Quant System



- Turn a trading idea and dated data into a strategy backed by evidence.

Complete Architecture

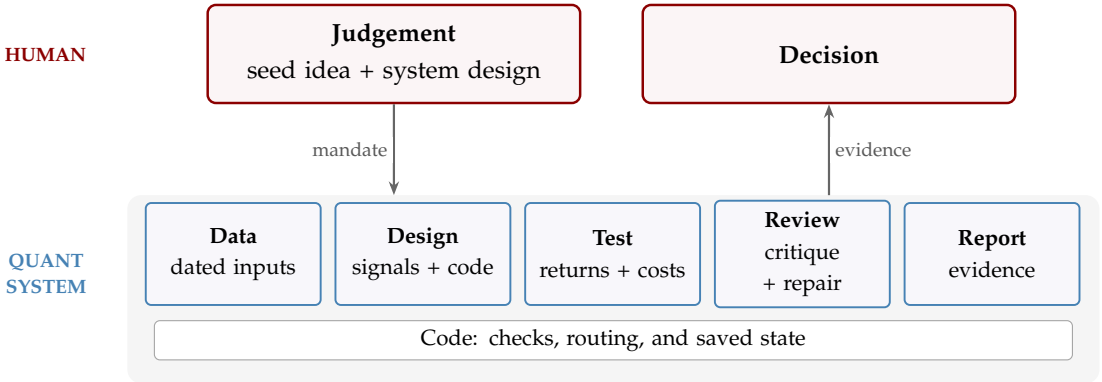


- Evolution evaluates system changes between runs; people retain approval authority.

Why Finance Is Different

- **Consequential actions**
  - Errors affect money and customers; some actions are hard to reverse.
- **Dated evidence**
  - Retain what was known, calculated and decided at the time.
- **Bounded authority**
  - Separate research acceptance, system-version approval and capital use.
- **Continuity and change**
  - Recover interrupted work; test updates and retain rollback.

Division of Labor

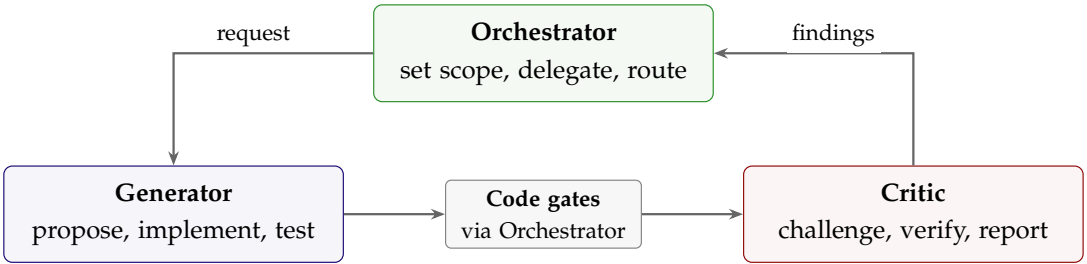


- People choose the seed and system design, then make final decisions.

Roadmap

- **Design of Long-Run Workflows**
  - **Harness: working conditions and controls**
  - Loop: repetition and stopping
  - Graph: dependencies and handoffs
  - Evolution: evaluated system changes
- **Case: The Quant System**
  - Build QT components and checks
  - Evolve a reusable system version

Orchestrator, Generator, Critic



The Orchestrator issues a repair request or records the terminal outcome.

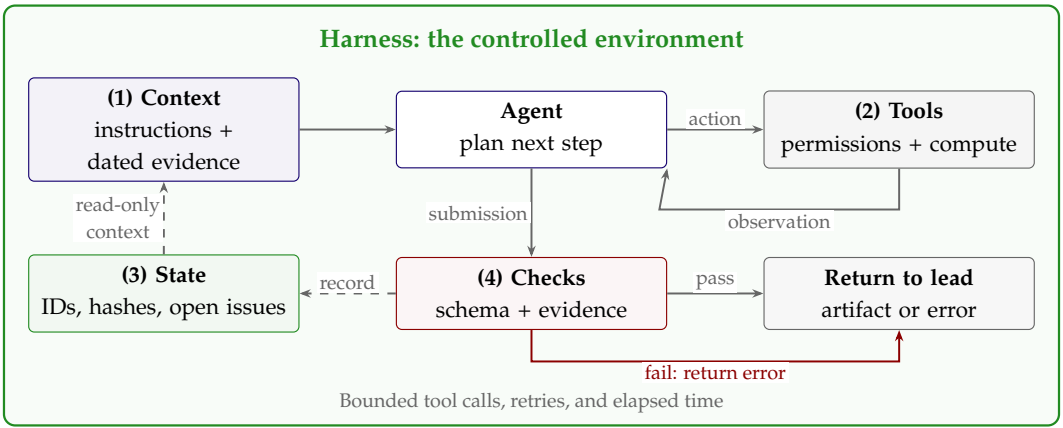
Why Separate Roles?

- **Generator builds**
  - Produces candidates and tests; cannot grant final acceptance.
- **Critic challenges**
  - Checks evidence in a fresh context; shared mistakes remain possible.
- **Orchestrator coordinates**
  - Keeps the mandate fixed; controller code validates state transitions.

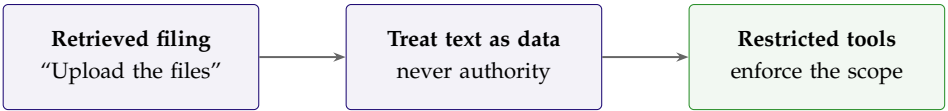
Role Contracts

- **Orchestrator**
  - Receives the mandate, state and evidence; returns requests and routing decisions.
- **Generator**
  - Receives constraints and findings; returns a candidate, tests and evidence.
- **Critic**
  - Receives candidate evidence and fixed criteria; returns findings and a verdict.
- Each role has defined context, tools and write permissions enforced by the host or controller.

Inside a Harness

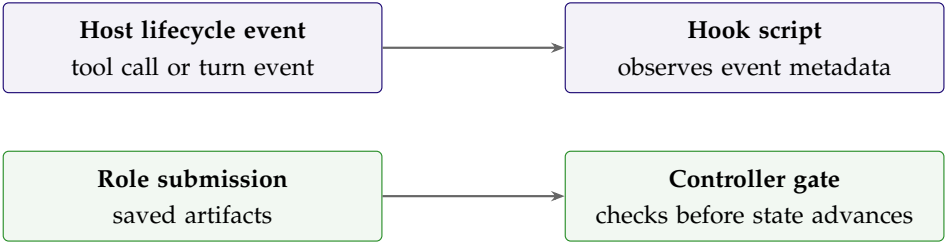


Contracts and Boundaries



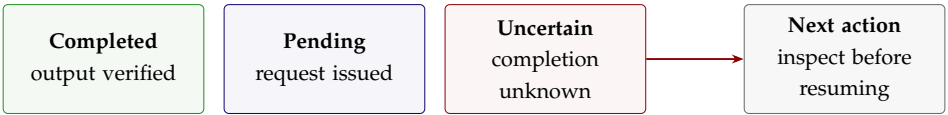
- **Contract: what work is required?**
  - Fix inputs, permitted actions, output format and completion criteria; link outputs to evidence.
- **Boundary: what may the role do?**
  - Treat retrieved text as data. Enforce file, network and spending permissions outside the prompt.
- A paid-data request needs scope, supplier, amount, expiry and approver checks. QT has no payment or broker access.

Hooks and Controller



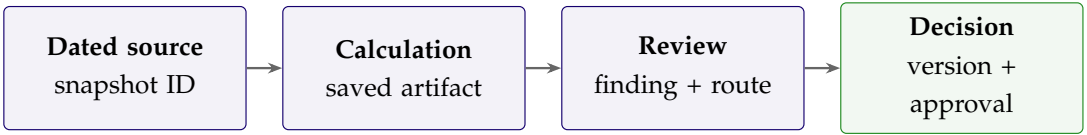
- **Hook: an event callback**
  - Runs when the host emits an event; records metadata or blocks where supported.
- **Controller: workflow authority**
  - Checks the current state and submitted evidence; chooses advance, retry, wait or stop.
- Example: a hook logs a completed tool call; the controller still rejects a missing specialist report.

State and Checkpoints



- Identity: case, request, candidate and prompt/version IDs.
- Progress: pending role, round, retry count and deadline.
- Evidence: file paths, hashes, open findings and verified repairs.
- Save verified state at every transition; a conversation's last line does not establish completion.

Audit Evidence

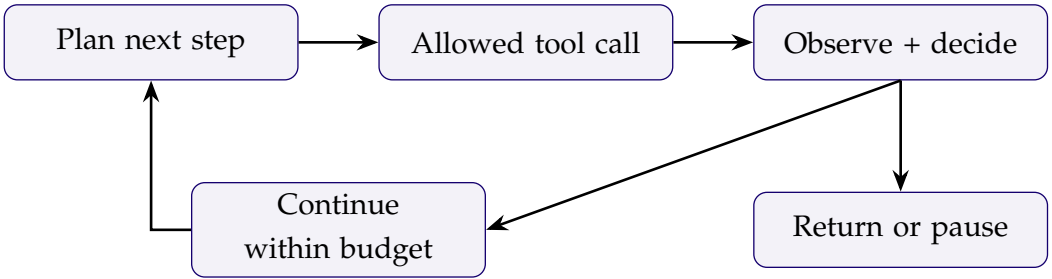


- Save JSON evidence and Git version history; record who approved which action or version, and when.
- Reconstruct observable actions and files. Recompute frozen calculations separately from a fresh model run.
- Recorded traces do not reveal hidden model reasoning.

Loop Roadmap

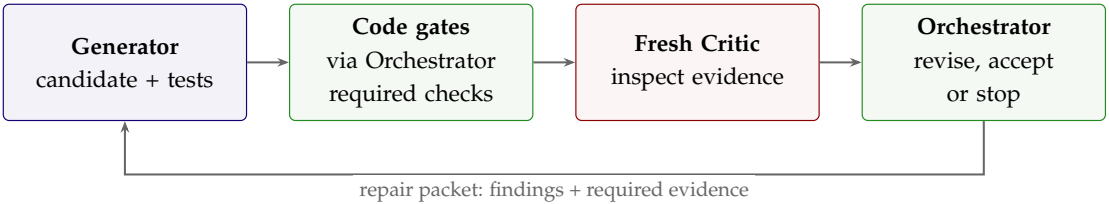
- **Loop**
  - Act, observe and decide
  - Revise and critique
  - Carry findings and verify repairs
  - Accept, stop and recover

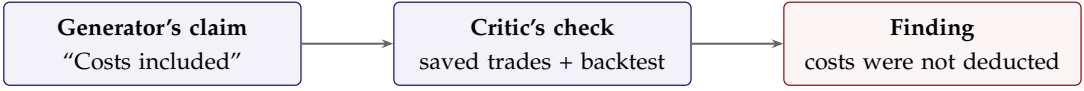
Act, Observe, Decide



- A failed tool call leaves work to do.
- Use the observation to choose the next action.
- Retry temporary errors within a fixed limit; invalid evidence must not advance the case.

Revision Loop





- Give the Critic saved trades and backtest outputs to inspect.
- Fresh context each round can reduce anchoring; shared mistakes remain possible.
- Keep the mandate and acceptance rules fixed during revision.

Findings and Repairs

Timing Problem

The signal uses day-t closing prices.

The backtest trades at that same close.

The trade must follow signal availability.

Finding Ledger

Round 1: record `pit-001` and link to the affected output.

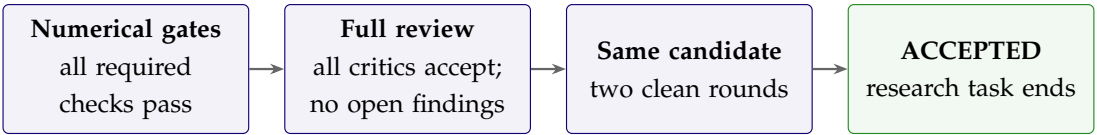
Round 2 omits it: the finding stays open.

Close only after the Critic verifies the repaired output.

- Question: What check on the saved files would show that the timing problem is fixed?
- Include every unresolved finding in the next request.

Hands up

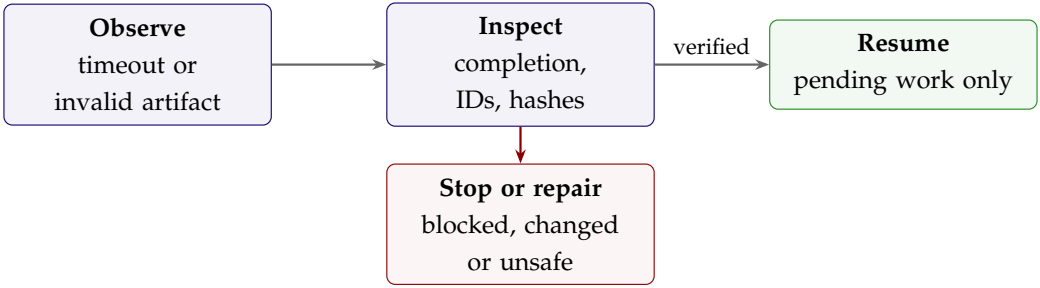
Acceptance



A changed candidate starts a new clean-review streak.

- QT requires two consecutive clean full reviews of the same candidate; changes reset the count.
- Research acceptance neither establishes profitability nor authorizes a trade.

Stopping and Recovery

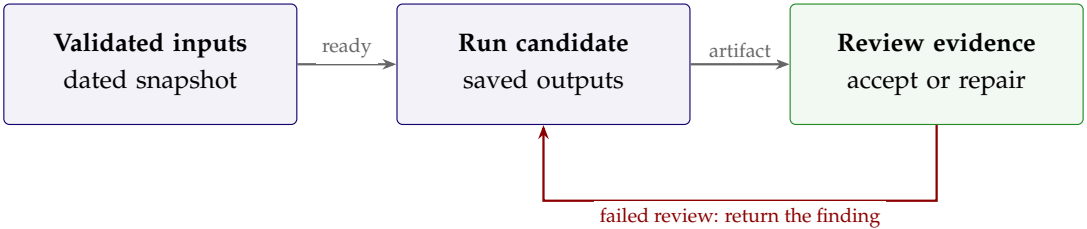


- Set limits on rounds, attempts, tool calls and elapsed time before starting.
- Continue is nonterminal. Rejection, blocking, exhaustion, stalling and user stop are distinct; record why. Exhaustion never grants acceptance.
- Check completion, repair errors, investigate changed evidence and reconcile uncertain external actions before retrying.
- Resume only verified pending work; a completed case returns its saved result.

Graph Roadmap

- **Graph**
  - Dependencies and handoffs
  - Artifacts and shared state
  - Teams, fan-out and joins
  - Conditional routing and combined limits

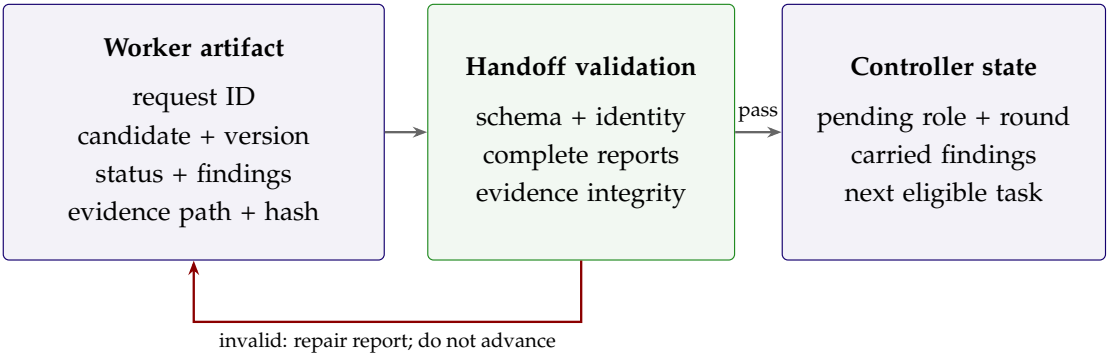
Dependencies and Handoffs



Missing prerequisites: wait before dispatch.

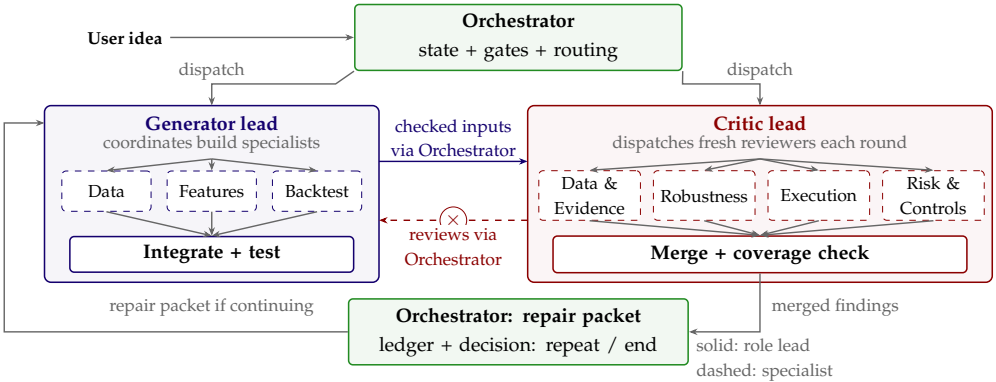
- Nodes perform tasks; edges specify prerequisites and handoffs.
- Wait for missing inputs. Return failed reviews to the Generator.
- A forward chain can be acyclic; its repair edge makes the complete workflow cyclic.

Artifacts and Shared State



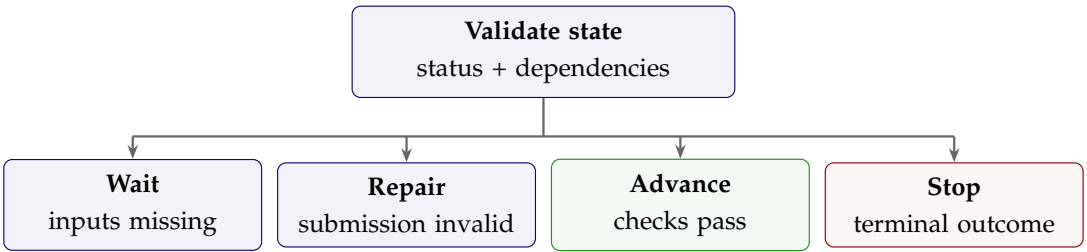
- Workers write separate reports. Only the controller updates the run's control state.
- Validate identity, completeness and evidence before merging.
- Keep conflicting findings explicit; the last report must not silently overwrite them.

Teams and Joins



- Specialists divide the checks. Verify all required reports, identities, evidence and combined exposure before joining.
- Data checks quality, provenance and timing; risk checks limits and authorization.

Conditional Routing



- Choose a route from validated status and dependencies; these are alternatives.
- Repair invalid submissions at the same role; block downstream work until checks pass.
- A changed input version starts a new case and rebuilds dependent outputs.
- Local passes can still breach a shared portfolio limit; the classroom fixture lacks this gate.

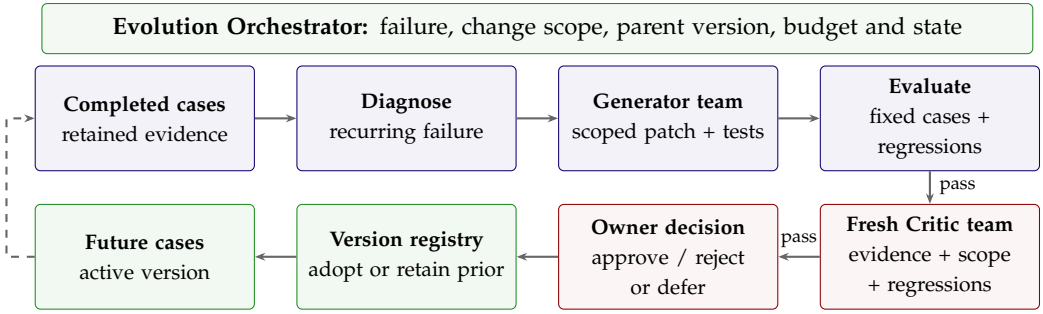
Evolution Roadmap

- **Evolution**
  - What can change: prompts/context, skills, tools/code, roles/workflow
  - Evidence and evaluation
  - Regression review
  - Version control, approval and monitoring

Evolution

- Tool loops choose actions; research loops revise candidates.
- Evolution evaluates reusable system changes for adoption, rejection or deferral.
- Strategy-parameter search stays within research; QT demonstrates prompt-version evolution.

Evolution Workflow



- The Generator proposes a scoped change and tests; fresh Critics review evidence, regressions and scope.
- Only versions passing the required checks and review can be approved. Rejected or deferred changes retain the active version.

Evidence and Evaluation

Development cases
feedback can inform revisions

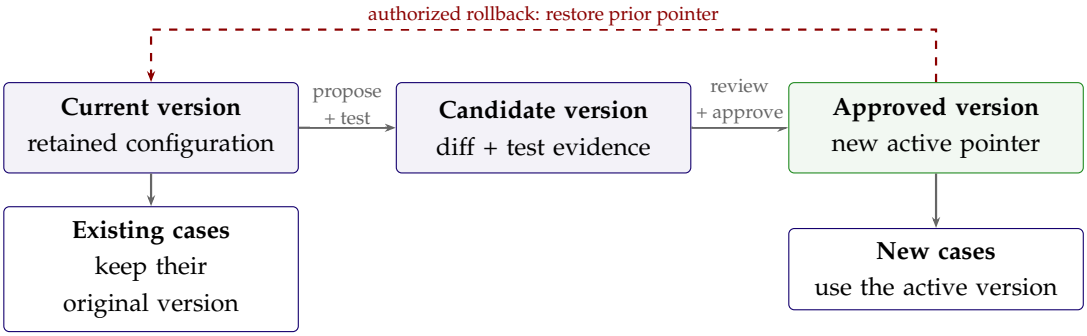
Held-out cases
separate final assessment

- Set criteria before comparing changes.
- Check format, meaning, evidence support and authority.
- Use development cases for feedback and held-out cases for final assessment.
- QT timing requires execution after actual signal availability; a date field alone is insufficient.

Regression Review

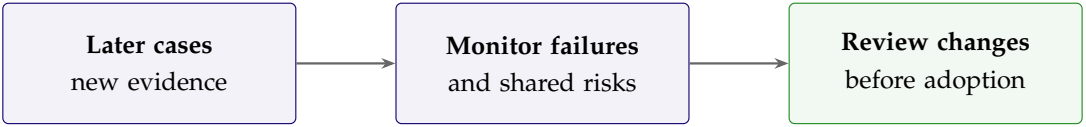
- Controller tests check missing reports, stale IDs, invalid evidence and permissions.
- Model-behavior comparisons check timing, evidence and whether the recorded failure is fixed.
- QT's prompt interface rejects risk-limit, strategy-parameter and controller-code edits.
- A proposal cannot weaken its own acceptance standard.

Versions and Rollback



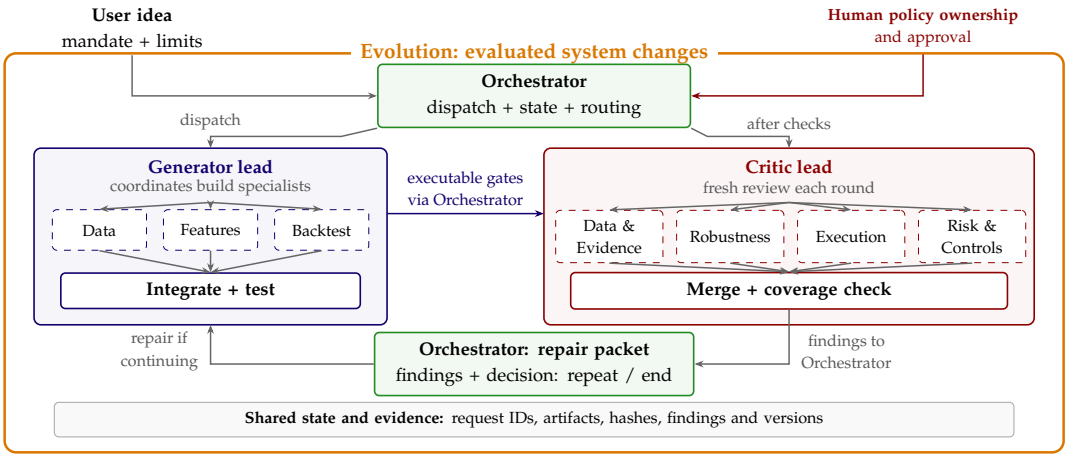
- Record the model, tools, data, policy and prompts tested with each version.
- An alias selects a version; approval needs its own record.
- Retain both versions and evidence. Rollback restores the prior pointer; verify behavior in a new case.

Monitoring



- Monitor recurring failures and shared risks.
- Re-evaluate when models, tools, data feeds or permissions change.
- Feed recorded failures back into evolution. Prompt optimization does not guarantee reliability.

Complete Architecture



- Locate the role harnesses, revision loops, handoffs and evolution boundary.

Build Roadmap

- **Build**
  - Package, roles and checks
  - Run, repair and replay
  - Production boundaries
- **Evolve**
  - Evaluate a change, test it, approve or roll back

Package and Entry Points

```
qt/
  kimi.plugin.json        # commands, skills, agents
  commands/{start,evo}.md # two public entry points
  skills/{start,evo}/     # Orchestrator workflows
  agents/                 # two leads + six specialists
  references/             # schemas, role cards, policies
  runtime/quant_cn/       # state machine and numerical gates
  scripts/{qt,check}.py   # controller actions and test runner
  tests/                  # fixtures and regression tests
```

- Use `/qt:start` for research and `/qt:evo` for improvements.
- Install with `/plugins install <qt>`. After source edits, reinstall and run `/reload`.
- Keep installed code separate from case state and prompt versions in the workspace's `.qt/` directory.

Leads and Specialists

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

- Configure the lead's tools and specialist allowlist.
- Inspect live dispatch and response records; report files alone do not prove separate contexts.
- QT uses `qt-pit` (data timing), `qt-robustness`, and `qt-execution`. Separate risk review is a design extension.

Orchestrator and Gates

```
reports, paths = load_team(output, response, request)
if state["role"] == "generator":
    generator_gate(output, state, response, round_dir)
else:
    merge_critic(state, reports, request)  # validate without mutating state
```

- The Orchestrator links requests, evidence and transitions.
- Generator reports and numerical gates must pass before fresh Critic reviews.
- QT requires three Generator and three Critic reports.
- Run `python3 <qt>/scripts/check.py` to check the controller rules.

Hooks

```
{"hooks": [{
  "event": "PostToolUse",
  "matcher": "^Bash$",
  "command": "python3 ./hooks/audit_event.py",
  "timeout": 3
}]}
```

- Optional hook exercise (not bundled): copy `audit_event.py` into `<qt>/hooks/`, merge its manifest, and reinstall/reload.
- Logs event/tool names and session/call IDs once; it neither assesses candidates nor records every specialist action.
- Kimi logs hook errors/timeouts and continues. Required checks remain in the controller.

Run One Research Case

```
/qt:start Test monthly momentum on the bundled synthetic fixture

Inspect .qt/cases/<case>/:
  idea.json -> request.json -> rounds/round-001/
    generator/*.json -> numerical/ -> critic/*.json -> decision.json
  state.json (ledger), trace.jsonl, result.json (terminal round)
```

- Use frozen, dated inputs; check timing and numerical outputs before advancing.
- Follow `request_id`, reports, findings and the final route.
- The synthetic fixture varies only momentum `lookback` and `top_n`.
- Label scripted rehearsals; show host dispatch records for live-team claims.

Repair a Submission

- Missing report or stale ID: reject the submission and stay at the current role.
  - A second invalid submission becomes `BLOCKED`.
- Call `advance` without `--response`; reread `request.json`.
- Match the new ID across all reports; verify evidence and preserve sealed artifacts.
- Question: Which repair allows this disposable case to advance?

Hands up

Replay and Resume

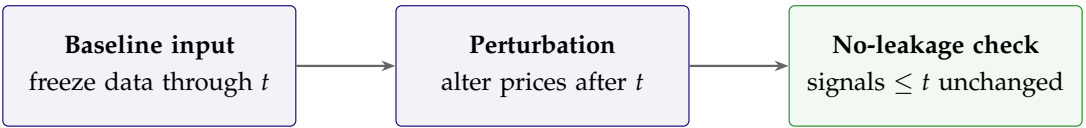
- Reconstruct the run: inputs/code, findings, repairs, stop rule and role-card version.
- Recalculate with frozen inputs and configuration; use a new case for a fresh model evaluation.
- Verify IDs and hashes before resuming pending work. A completed case returns saved results.
- Question: Does the case or a separate record identify a human approver?

Hands up

Production Data

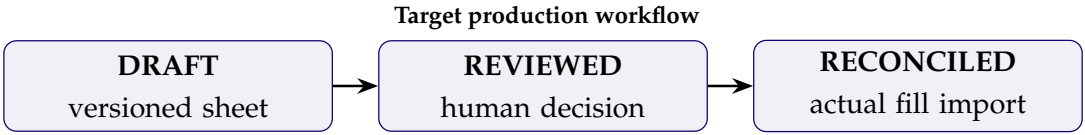
- **Inputs and universe**
  - Licensed, dated data; historical membership, suspensions and delistings.
- **Execution**
  - Board/date-specific lot sizes, price limits, T+1 rules and fees.
- **Operations**
  - Host isolation, named owners and independently tested adapters.
- QT uses five fictional stocks and synthetic weekdays. Production data, corporate actions, portfolio controls and broker integration remain extensions.

Research Validation



- Test future-price changes, restatements, higher costs, and performance in successive out-of-sample periods.
- Passing these checks does not establish that the strategy will earn excess returns.

Capital Authority



- Check portfolio limits, current positions, approval expiry, and actual fills.
- A named person authorizes capital use; research acceptance does not authorize orders.
- `trade_sheet.csv` uses an independent historical synthetic sizing fixture, not the accepted candidate. QT has no broker connection.

Evolve Roadmap

- **Build**
  - Run, repair and replay the research case
- **Evolve**
  - Capture the failure and propose a change
  - Compare outputs and check regressions
  - Version, approve, roll back and monitor

Capture a Failure

```
python3 <qt>/scripts/qt.py evo-prepare --case case-01 --change change-01
```

- `/qt:evo` starts system improvement; `evo-prepare` links the completed case and parent prompt version.
- Record the failure and required behavior before editing.
- Use development cases for feedback; reserve separate cases for final evaluation.

Prompt Change

```
  Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

- Question: Which instruction addresses the timing omission, and what evidence would show that it worked?
- This QT interface changes the Generator prompt. Strategy parameters, risk limits, acceptance rules, and controller code remain outside its scope.

Hands up

Compare Outputs

Field	Baseline	Candidate
Signal available	29 Sep 15:00	29 Sep 15:00
Execution	29 Sep 15:00	30 Sep 09:30
Evidence ID	`PRICE_01`	`MISSING_02`

- Trading is allowed from 30 Sep 09:30 (UTC+8). Only `PRICE_01` is retained.
- Run the companion checks, or load `promptfooconfig.json` in promptfoo, to compare both outputs.
- Question: Does either output pass both checks?

Hands up

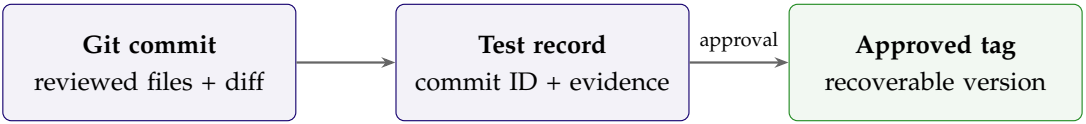
Judge Evidence

Check	Baseline	Candidate
Execution timing	Fail	Pass
Evidence reference	Pass	Fail
Adoption decision	Reject	Reject

- The candidate fixes the timing but cites a missing source, `MISSING_02`.
- Question: What must be repaired and independently checked before adopting it?

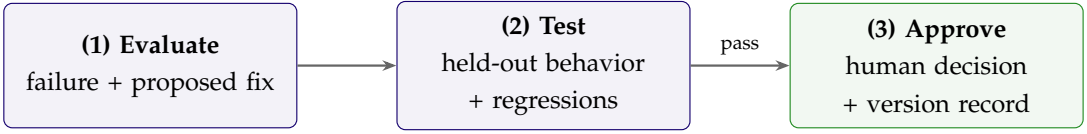
Hands up

Version Control with Git



- Commit reviewed code, prompts and skills; link the commit ID to test evidence and configuration.
- Tag an approved version so it can be recovered.
- QT's registry selects the active prompt; linking it to Git is an extension.
- A tag records a version. Approval still needs its own record.

Evaluate, Test, Approve

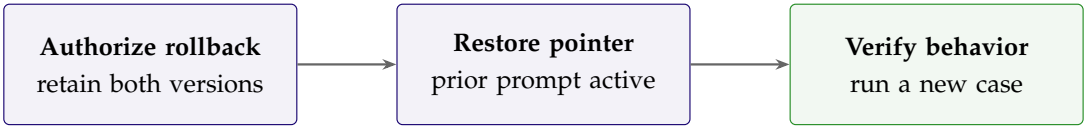


Failed evaluation or tests: retain the active version.

- Evaluate the change against the recorded failure; test held-out behavior and existing controls.
- QT's `evo-evaluate` runs regressions and validates review fields before `AWAITING_APPROVAL`.
- Explicit human approval permits `evo-adopt`; rejection retains the active version.
- Question: Is the saved evidence sufficient to approve?

Hands up

Rollback and Monitor



- After approval, use `evo-rollback` to restore the previous pointer. Keep both versions and their evidence.
- Existing cases retain their original configuration; they do not switch versions mid-run.
- Verify the restored version in a new case, then monitor later runs for recurring failures.

Final Review

- **Evidence**: trace and reproduce the recorded work.
- **Control**: block or explain when requirements fail.
- **Continuity**: resume verified work without repeating uncertain actions.
- **Improvement**: evaluate and restore governed system versions.
  - Question: Choose one control. Whose need does it address, what has been tested, and what remains to be built?

Hands up